3 블록 설정, 다중창 및 단축키 지정

지금까지 배워온 vi 에디터, 이미 다른 편집기에 손이 익은 사람이라면 쉽지 만은 않았을 것이다. 하지만 이것만은 알아두자! vi 에디터는 손에 익으면 익을수록 몸이 덜 고생하는 편집기이다. 지금까지 사용한 대부분의 명령어들이 화살표나 조합키를 사용하는 경우가 드물기 때문에 손가락이 온 자판을 휘젓고 다닐 필요가 없기 때문이다. 이번 차 연재에서는 필요한 부분을 복사해서 붙여 넣을 수 있는 블록과 여러 파일을 열고 편집할 수 있는 다중창 그리고 단축키를 지정하는 방법에 대해 알아보도록 하겠다.

1. 블록 설정

이번 단원에서는 특정한 구간을 설정하여 삭제하거나 복사하는 기능을 소개하고자 한다.

1.1 붙여넣기

'x' 나 'dd' 와 같은 삭제 명령 후에 삭제된 데이터들은 완전히 없어지는 것이 아니라 vim 의 메모리에 저장되어 있어 원하는 위치로 커서를 이동시킨 후 붙여 넣기를 할 수 있다.

삭제된 문자나 줄을 붙여 넣을 수 있는 명령은 'p' 이다. 다음은 'p' 명령으로 삭제된 데어터를 붙어 넣는 장면이다. 먼저 please 의 첫 문자에 커서가 놓인 상태에서 3x 명령으로 'ple' 3개의 문자를 삭제하고 4j 명령으로 오른쪽으로 4 문자 이동하였다. don't 의 첫 문자에 커서가 놓여있을 때 'p' 명령을 내리면 현재 커서가 위치한 곳 다음에 삭제된 'ple' 세 문자를 붙여 넣기를 실시한 것을 알 수 있다.



'p' 명령은 삭제된 내용을 커서 다음에 붙여 넣었다. 반대로 대문자 'P' 명령은 커서 앞에 삭제된 내용을 붙여 넣는다.



'P' 명령 이후 d 앞에 ple 가 붙여짐

삭제된 내용을 붙여넣는 것은 다른 에디터의 Cut&Paste 와 비슷한 기능이다. Cut 이 아닌 Copy&Paste를 하기 위해서는 Vim의 'Yank' 기능을 이용해야 한다.

Yank 는 'y' 명령을 이용하고 사용법은 삭제 명령인 'd' 와 비슷하다. 먼저 한 단어를 복사하기 위해서는 'yw', 한 줄을 복사하기 위해서는 'yy'를 사용한다. 물론 여러 단어 혹은 여러 줄을 복사하기 위해서는 앞에 원하는 만큼의 숫자를 적어 주면 된다.



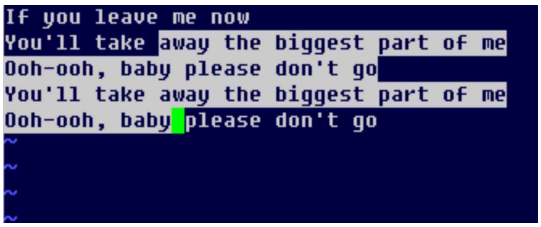
2yy 로 2 줄을 복사하고 밑으로 한 이동 한 후
p 명령으로 붙여 넣기

1.2 마크

위에서 문자, 단어, 줄 단위로 Cut&Paste 및 Copy&Paste를 해보았다. 하지만 여러 줄과 단어에 걸쳐있는 구간을 원하는 대로 설정하기 위해서는 vi의 마크 기능을 이용해야 한다.

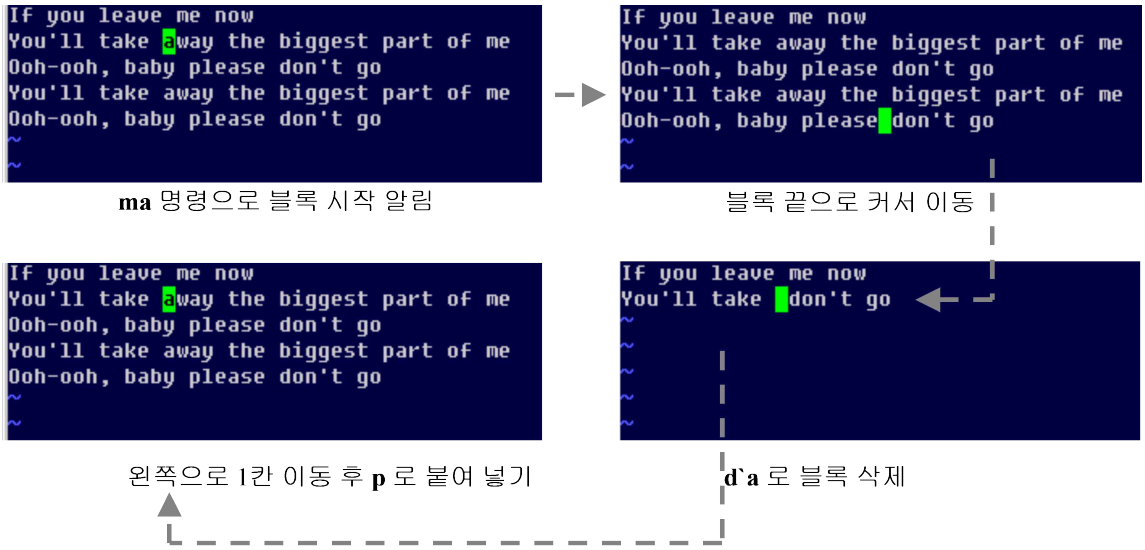
마크란 a~z 사이 임의의 한 문자에 현재의 위치를 기억시키는 것이다. 매크로에서 알파벳 a~z 중 한 문자를 지정했던 것을 떠올리면 이해하기가 쉬울 것이다. 마크를 사용하는 방법은 기억시키고 싶은 위치에 커서를 옮긴 후, 먼저 'm'을 누른 후 a~z 임의의 알파벳 하나를 누르면 된다. 만약 'a' 라는 문자에 현재의 위치를 기억시키고자 하면 'ma'를 연달아 입력하면 된다. 알파벳에 위치를 기억시키기 때문에 총 26 곳의 위치를 저장할 수 있다. 그리고 일단 'a' 라는 문자에 위치를 기억시킨 후 커서를 다른 곳으로 옮긴 상태에서 **'a(작은 따옴표가 아니라 숫자 '1' 왼쪽에 있는 키)**를 누르면 a 가 기억하고 있던 위치로 커서가 옮겨진다.

그럼 마크를 이용하여 다음 구간을 Cut&Paste 해보도록 하자.



Cut&Paste 할 영역

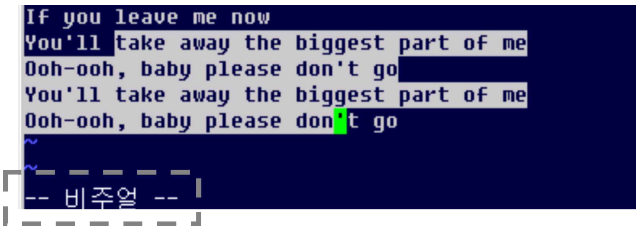
먼저 삭제하고자 하는 곳의 시작 위치로 커서를 이동한 후 'ma'를 누른다. 그리고 삭제하고자 하는 곳의 끝 위치로 이동한 후 'd`a'를 누르면 원하는 구간의 내용이 삭제 될 것이다. 이제 붙여넣기 하고자 하는 쪽으로 커서를 옮긴 후 'p' 명령을 내려주면 된다.



이번에는 마크를 이용하여 Copy&Paste를 해보자. 삭제할 때와 똑같이 시작 위치에서 'ma'를 누른다. 이제 끝 위치로 이동하여 'y`a'를 누른다. 삭제 때와 달라진 것은 d 가 y 로 변한 것 밖에 없다. 끝으로 원하는 위치에 가서 'p' 명령을 내리면 된다.

1.3 Visual Mode

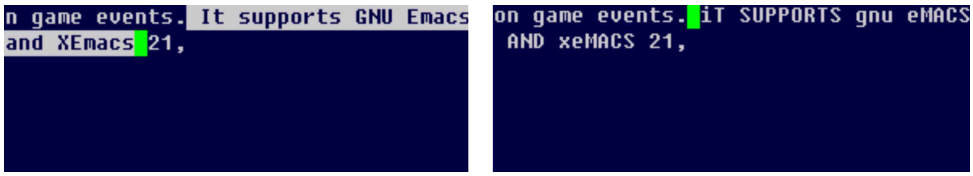
vim은 다른 vi 에디터와 달리 Visual Mode 라는 것이 있다. 마크를 사용한 블록 설정은 설정한 구간에 시각적인 효과를 주지 않기 때문에 간혹 구간 설정이 제대로 되었는지 모르는 경우가 있다. Visual Mode 는 바로 Visual Block 으로 구간에 시각적인 효과를 준다.



Visual Block을 생성하는 방법은 간단하다. "Normal Mode"에서 '**v**' 명령을 내리면 바로 "Visual Mode" 로 들어간다. 구간 설정을 위해 Visual Mode 로 들어가려면 먼저 블록을 잡고 싶은 시작 위치로 가서 'v'를 입력한다. 그러면 바로 Visual Mode 로 들어가게 되는데, 이때 커서를 블록의 끝 위치로 이동시키기만 하면 Visual Block 이 완성된다. 블록이 생성된 상태에서는 삭제나 복사를 할 수 있다. Visual Block 이 생성된 Visual Mode 상태에서 구간을 삭제하고 싶다면 'd' 나 'x'를 누르면 구간내의 내용이 사라지고 바로 "Normal Mode" 로 상태가 바뀌게 된다. 만약 Visual Block 이 생성된 Visual Mode 상태에서 구간을 복사하고 싶다면 'y'를 누르면 된다. 이제 남은 일은 붙여넣기를 하고 싶은 위치로 커서를 이동 시킨 후 '**p**' 명령만 내려주면 된다.

"Visual Mode"를 빠져나가 "Normal Mode" 로 돌아가기 위해서는 다른 Mode 와 마찬가지로 <ESC>를 누르면 된다.

"Visual Mode"를 사용하면 블록 내에 있는 모든 문자의 대소문자를 변경 시킬 수 도 있다.



Visual Block을 이용한 대소문자 변경

먼저 원하는 구간을 Visual Block 으로 잡은 뒤 대소문자 변경 명령인 '~' 명령만 내려주면 된다.

2. 여러 파일 편집

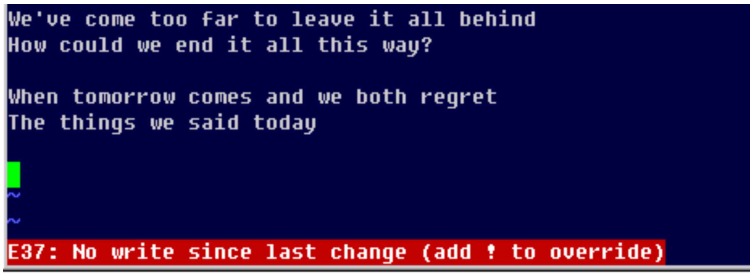
문서를 편집하다 보면 다른 파일로부터 내용을 발췌하는 일이 빈번하다. 이 때 파일 여러 개를 동시에 열어두고 편집하면, 다른 파일로부터 내용을 복사해 현재 편집중인 파일에 붙여 넣기가 편할 것이다.

파일을 동시에 여러 개 열고 싶다면, 단지 vim을 실행할 때 원하는 파일들의 이름을 인자로 넘겨주면 된다.

$ vim test.txt test2.txt

2.1 편집 파일 변경

$ vim test.txt test2.txt 처럼 vim을 실행할 때 2개 이상의 파일을 열게 되면 일단 첫 번째 파일인 test.txt 가 화면에 나타나게 된다. 두 번째 파일을 편집하기 위해서는 현재 편집 중인 파일을 :w 로 저장한 후 :next 명령을 내리면 된다. 만약 저장하지 않고 다음 파일을 편집하려 하면



위와 같이 변경된 후 저장하지 않았다는 에러 메시지를 출력한다. 위의 메시지에서도 알 수 있듯이 만약 저장하지 않고 다음 파일로 넘어가고 싶다면 :next! 와 같이 !를 추가하면 된다. 저장과 다음 파일로 전환을 동시에 할 수도 있다. :wnext 명령은 현재 파일이 변경되었으면 저장한 후 다음 파일을 편집할 수 있도록 한다.

계속해서 :next 명령으로 다음 파일로 넘어가다 마지막 파일까지 오게 되면 더 이상 :next 로 이동할 수 없게 된다. 이전 파일로 가는 명령은 :Next 혹은 :prev 이다. 비슷하게 저장하고 이전 파일로 가기 위해서는 :wprev 혹은 :wNext 명령을 내리면 된다.

현재 파일에서 다음 파일로 넘어가기 위해서는 미리 저장을 하거나 :wprev, :wnext 같은 명령을 써야만 했다.

하지만

:set autowrite

명령을 내려주면 :next 나 :prev 로 파일을 전환할 때 자동으로 파일을 저장해 준다. 자동 저장 기능을 해제하기 위해서는

:set noautowrite

명령을 내려주면 된다.

이렇듯 파일 사이를 전환하다 현재 무슨 파일을 편집하고 있는지 알기 위해서는

:args

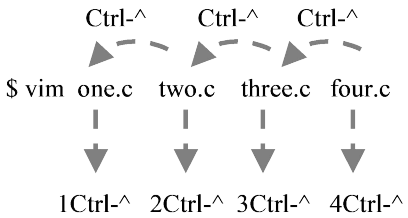
명령을 사용한다.

:next 나 :prev 는 vim을 기동할 때 나열했던 파일의 순서대로 편집 위치를 이동시킨다. 나열했던 파일들 하나씩 하나씩 이동하지 않고 처음 파일 혹은 나중 파일로 한번에 이동하기 위해서는

:rewind
:last

명령을 사용하면 된다. :rewind 는 처음 파일, :last 는 마지막 파일로 이동을 시킨다.

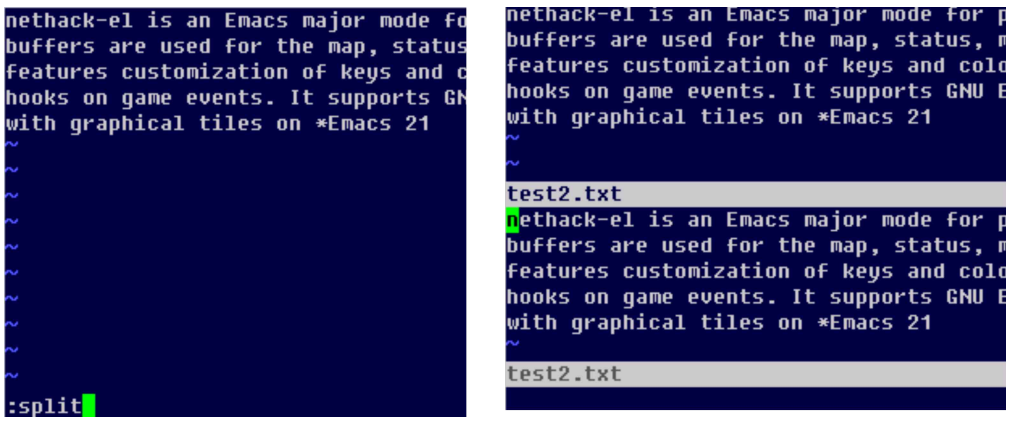
이런 방법 외에도 다음 Ctrl-^를 사용 할 수 있다. 다음 그림처럼 one.c two.c three.c four.c 총 네 개의 파일을 열 경우 첫 번째 파일로 바로 가기 위해서는 1Ctrl-^, 두 번째 파일로 바로 가기 위해서는 2 Ctrl-^ 명령을 내리면 된다. 그리고 이전 파일로 가기 위해서는 숫자 없이 Ctrl-^ 명령만 내려주면 된다.



2. split

지금까지는 한꺼번에 여러 개의 파일을 열 수 있었지만 실질적인 편집은 한 창에 한 파일씩 진행하였다. 만약 한 창에 여러 개의 파일을 동시에 열어 놓고 편집하고 싶다면 창을 분할해야 한다.

창을 분할하기 위해서는
:split
명령을 사용한다. :split 으로 창을 분할하면 다음과 같이 가로로 창이 나뉘어 진다.



:split 으로 창 나누기

단순히 :split 명령만 내리면 위와 같이 같은 파일을 분할 된 두 창 모두에서 편집하게 된다. 분할과 동시에 새로운 파일을 열고자 한다면

:split 파일이름

과 같이 파일이름을 동시에 적어주거나 이미 :split 으로 분할이 된 상태라면

:e 파일이름

명령으로 새로운 파일을 불러오면 된다.
위와 같이 창이 분할 된 상태에서 다른 창으로 이동 하기 위해서는

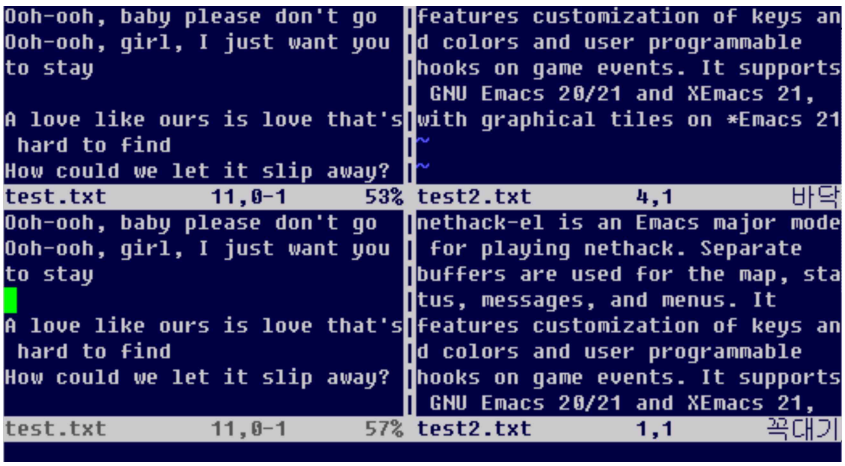
Ctrl-ww (Control 키를 누른 상태에서 w를 두 번 입력한다.) 명령을 내리면 된다.

창을 위와 같이 수평으로 나눌 수도 있지만 다음 명령으로 수직으로 나눌 수도 있다.

:vsplit
:vsplit 파일이름

split 앞에 v를 붙여주면 수직으로 창을 분할한다. 이미 분할 된 창에서도 :split 이나 :vsplit 으로 얼마든지 창을 더 분할 할 수 있다.

다음 그림은 :split 으로 분할된 창을 다시 :vsplit을 이용하여 위아래 창 모두 수직으로 분할 한 모습니다.



창을 총 4등분한 모습

물론 창이 아무리 많이 분할되더라도 창과 창 사이는 Ctrl-ww 명령으로 이동 할 수 있다. 만일 창이 굉장히 많이 열려있다면 (물론 이렇게 많은 창을 열고 편집하는 사람은 없을 것이다.), 창 하나하나를 Ctrl-ww 명령으로 이동하는 것이 무척 지루한 작업 일 수도 있다. 이때

Ctrl-wh, Ctrl-wj, Ctrl-wk, Ctrl-wl (Ctrl-w 와 방향을 나타내는 h,j,k,l 키를 조합한 것이다.)

명령을 사용하면 상,하,좌,우 원하는 방향으로 창을 이동할 수 있다.

창을 닫는 명령은 간단하다. 우리가 흔히 아는 방법인 :q, :q! 로 커서가 위치한 창을 닫을 수도 있고

Ctrl-Wc

명령으로 창을 닫을 수 있다.

3. 단축 과 Key Mapping

3.1 단축

편집을 하다보면 같은 단어가 반복되는 경우가 많다. 그런데 그 단어라는 것이 매우 길거나 익숙하지 않은 특수 문자가 섞여 있다면 반복되는 지루한 작업에 염증을 느끼게 될 것이다. 이러한 단어들이 편집자의 생각대로 자동으로 타이핑이 된다면 몸과 마음이 얼마나 편하겠는가? vi 는 자주 반복되는 단어를 짧은 몇 개의 문자에 등록 한 후 자동으로 단어가 완성되는 단축 기능을 제공한다.

단축 기능의 사용법은 다음과 같다.

:abbreviate va variables

위에서 "va" 는 단축어이고 "variables"는 완성될 단어를 뜻한다. 위와 같이 등록 한 후 "Input Mode"에서 “va” 까지만 입력한 후 <SPACE>, <ENTER>, <TAB> 과 같이 단어 사이를 분리해주면 va 가 variables 로 변경된다. 만약 "var" 까지 입력하면 "variables"로 변하지 않는다.

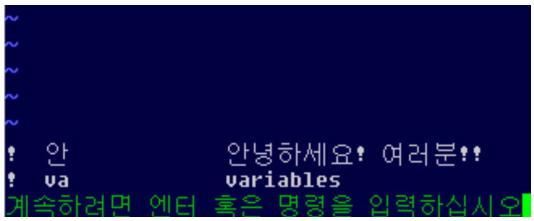
단어뿐만 아니라 다음과 같이 공백이 있는 문장도 단축어에 등록 할 수 있다.

:abbreviate 안 안녕하세요! 여러분!!

이제 “안” 이라고 입력한 후 <SPACE>를 누르면 “안녕하세요! 여러분!!” 으로 변할 것이다.

지금까지 등록된 단축어를 모두 보고 싶으면 :abbreviate 라는 명령을 주면 된다.

:abbreviate



:abbreviate 결과

한가지 주의할 점은 단축 기능은 명령어에도 적용이 된다는 것이다. 우리가 자주 사용하는 명령 중 :set 명령이 있다. 만약 "set"을 단축어로 등록하면 어떻게 될까?

:abbreviate set setting

이 상태에서 우리가 :set 명령을 입력하면 아주 곤혼스러운 결과가 생길 것이다.

:set - - - - - - → :setting
이 상태에서 <SPACE> 입력

위와같이 명령을 내리기 위해 ":set" 까지 입력 한 후 <SPACE>를 누르게되면 바로 :setting 으로 바뀌기 때문에 정상적인 명령을 내릴 수 없게 된다.

물론 이와 같은 단축어를 등록하지 않는 것이 가장 최상의 방법이겠지만, 혹시라도 이런 일을 당하게 되면 다음과 같이 단축어를 삭제할 수 있다.

:unabbreviate set

명령어가 하나 둘 늘어나면서 익히고 외우기가 무척이나 힘이 들 것이다. VIM을 위해 모든 명령을 완벽히 외울 필요는 없다. 첫부분만 알고 있어도 자동완성 기능으로 명령을 내릴 수 있기 때문이다. 이번엔 명령어를 완벽히 입력하지 말고 다음처럼 "**:a**" 만 입력한 상태에서 <TAB>을 눌러보도록 하자.

:a<TAB>

위 상태에서 <TAB> 여러 번 누르게 되면 그때마다 다음처럼 명령어가 변경되는 것을 확인 할 수 있을 것이다.

:append
:abbreviate
:abclear
...
...

만약
:ab<TAB>

과 같이 미리 알고 있는 처음 문자수가 많다면 그만큼 쉽게 명령어를 찾아 낼 수가 있다.

3.2 Key Mapping

Vim 에서는 Key Mapping을 통해 여러 명령을 하나의 Key로 실행할 수 있다. 전에 언급했던 매크로와 기능상 아주 유사하지만 정의 방법과 명령 실행 방법이 약간 다르다.

Key Mapping을 정의하는 명령은 다음과 같다.

:map <F5> 명령들

<F5>를 눌렀을 때 한 행을 복사하고 붙여넣기를 할 수 있는 Key Mapping 은 다음과 같이 정의 할 수 있다.

:map <F5> <ESC>yyp

위의 <F5> 의 경우 키보드의 Function Key를 직접 눌러도 되고 일일이 <F5>를 문자 그대로 입력해도 된다.

Key 매핑을 해지하는 명령은 다른 명령과 비슷하게

:unmap <F5>

처럼 할 수 있고, 정의 되어 있는 map 의 리스트는

:map

명령으로 확인 할 수 있다.

4. 설정 저장하기

지금까지 여러 명령을 실행해 보았다. 실행한 명령 중에는 현재 진행 중인 편집 뿐만 아니라 추후에도 자주 사용하게될 명령도 있을 것이다. Key Mapping 이나 단축같은 경우 좋은 예라 할 수 있을 것이다. 이러한 명령들은 다음과 같은 파일에 저장할 수 있다.

$HOME/.vimrc
$HOME/_vimrc
$HOME/.exrc
$HOME/_exrc

Vim이 기동 될 때 위의 파일을 순서대로 검사하며 4개의 파일 중 제일 먼저 읽히는 파일의

내용만 적용하게 된다. 만약 $HOME/.vimrc 파일과 $HOME/.exrc 파일 2개가 동시에 존재한다면 $HOME/.vimrc 파일만 적용 될 것이다.

현재 편집중인 vim 의 모든 설정을 .vimrc 파일에 저장하려면

:mkvimrc 명령을

.exrc 파일에 저장하려면

:mkexrc 명령을 입력하면 된다.

만약 $HOME 디렉티리에 이미 파일이 존재한다면 저장할 수 없다. 이때는

:mkvimrc 파일이름 혹은
:mkexrc 파일이름

명령을 사용할 수 있으며, 이러한 파일은 vim 이 기동될 때 자동으로 적용되지 않기 때문에 적용하기 위해서는

:source 파일이름

명령을 사용하면 된다.

$HOME/.vimrc 나 $HOME/.exrc 파일에는 지금껏 사용해왔던 명령들이 나열될 수 있다. .vimrc 파일에 다음과 같은 명령을 나열해 주면, 기동될 때마다 자동으로 명령들이 적용 될 것이다.

:abbreviate va variables
:map <F5> <ESC>yyp
:set hlsearch
:set incsearch